Available online at www.sciencedirect.com
SCIENCE DIRECT®
Applied Soft Computing 6 (2006) 139–153
Applied Soft Computing
www.elsevier.com/locate/asoc

Robust incremental growing multi-experts network

Chu Kiong Loo [a,b,1,*], Mandava Rajeswari [b,1], M.V.C. Rao [a,2]

[a] Faculty of Engineering and Technology, Multimedia University (Melaka), 75450 Melaka, Malaysia
[b] School of Industrial Technology, University Sains Malaysia, Penang, Malaysia

Received 11 December 2003; received in revised form 18 November 2004; accepted 17 December 2004

Abstract

Most supervised neural networks are trained by minimizing the mean square error (MSE) of the training set. In the presence of outliers, the resulting neural network model can differ significantly from the underlying model that generates the data. This paper outlines two robust learning methods for a dynamic structure neural network called incremental growing multi-experts network (IGMN). It is convincingly shown by simulation that by using a scaled robust objective function instead of the least squares function, the influence of the outliers in the training data can be completely eliminated. The network generates a much better approximation in the neighborhood of outliers. Thus, the two proposed robust learning methods namely robust least mean squares (RLMSs) and least mean log squares (LMLSs) are insensitive to the presence of outliers unlike the least mean squares (LMSs) cost function. Moreover, various types of supervised learning algorithms can easily adopt LMLS, which is a parameter-free method.
© 2005 Elsevier B.V. All rights reserved.

Keywords: Outliers; Dynamic structure neural network; Robust learning

1. Introduction

In the framework of function approximation using neural architecture, robust learning methods need to be used, when outliers are present in the given training data set. Otherwise the approximation realized by the neural architecture trained by a conventional learning method can suffer from substantial deterioration due to the outliers. In traditional MLP and RBF networks, the least squares (LSs) is selected as the objective function for network learning. A network adjusts iteratively parameters of each node by minimizing the LS criterion according to gradient descent algorithm. However, when some of the training data encounter large errors resulting from the presence of outliers, the network will yield improper responses in the neighborhood of outliers due to the LS criterion. In this regard, a robust backpropagation learning algorithm for MLP [7] and RBF [4,5] was proposed. In this regard, this paper revisits a novel type of dynamic structure network, incremental growing multi-experts network (IGMN) or also known as sequential growing multi-experts

* Corresponding author. Tel.: +60 6 2523322; fax: +60 6 2316552.
E-mail addresses: ckloo@mmu.edu.my (C.K. Loo), mandava@cs.usm.my (M. Rajeswari).
[1] Tel.: +60 4 6577888/2157.
[2] Tel.: +60 6 2523322; fax: +60 6 2316552.

1568-4946/$ – see front matter © 2005 Elsevier B.V. All rights reserved.
doi:10.1016/j.asoc.2004.12.003

network that has been proposed and studied recently [1–3,17]. In this paper, two robust learning scheme similar to the work in [5,12] are incorporated into IGMN network. Robust learning refers to a learning method capable of dealing with data containing outliers. In this learning scheme, the LS criterion is replaced by a scaled robust objective function. During the learning process, the outliers are detected and their negative influence is removed automatically.

2. Incremental growing multi-experts network

Overall, IGMN consists of three main building modules namely, expertise domain, expertise level, and local experts. Expertise domain is used to define the influence regions where the local experts perform local approximation. The competency of local experts is indicated by the expertise level. High value of expertise level means the local expert has significant contribution to the overall network output. The local expert models are gated by expertise level and expertise domain module as shown in Fig. 1.

2.1. Expertise domain

Gaussian basis function is chosen to define the expertise domains because of their highly desirable local properties. Each basis function's center, $\vec{c}_k$ is a point in the input space, and the receptive field (influence region) of the unit is defined by its distance metric $d(\vec{x}, \vec{c}_k, \vec{\sigma}_k)$. The Gaussian basis functions are composed of two elements. The distance metric $d(\vec{x}, \vec{c}_k, \vec{\sigma}_k)$ for Gaussian basis function k as defined in Eq. (1) can scale the spread of the Gaussian basis function relative to its center $\vec{c}_k$ and the Gaussian basis function takes the distance metric as input and $\vec{\sigma}_k$ as diagonal matrix:

$$d(\vec{x}, \vec{c}_k, \vec{\sigma}_k) = |\vec{x} - \vec{c}_k|^{\mathrm{T}} \vec{\sigma}_k^{-1} |\vec{x} - \vec{c}_k| \quad (1)$$

$$\mu_k(d(\vec{x}, \vec{c}_k, \vec{\sigma}_k)) = \exp(-d(\vec{x}, \vec{c}_k, \vec{\sigma}_k)) \quad (2)$$

Gaussian basis functions have localized receptive fields, for they decrease monotonically from a maximum at $\vec{x} = \vec{c}$ (distance metric = 0) toward zero. With this property, the Gaussian basis function is a suitable choice as an expertise domain gating function to indicate the validity of input data with respect to the local expert model. The validity value decreases as the inputs move away from the Gaussian basis function center. The value of all expertise domains ranges from 0 to 1. Fig. 2 depicts the representation of Gaussian function as expertise domain for IGMN.

2.2. Expertise level

When two expertise domains (Gaussian basis functions) are located close to each other, there will be a high degree of overlapping of the expertise domains. As a result, one of the local experts associated with the expertise domains might appear to be redundant. In such a case, single local expert is

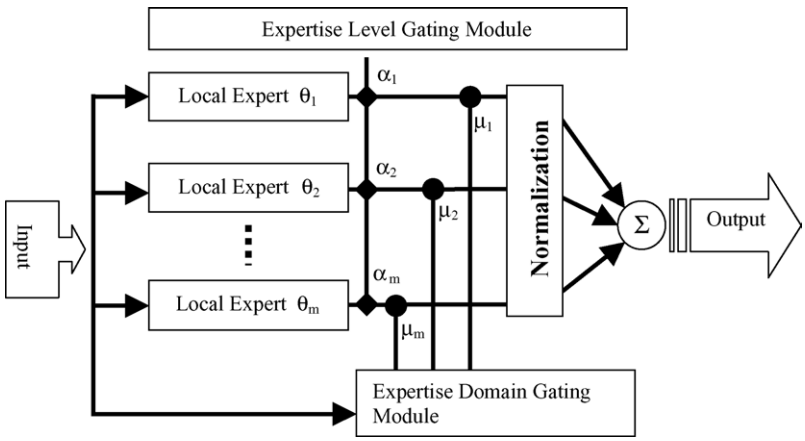

Fig. 1. Incremental growing multi-experts network architecture.

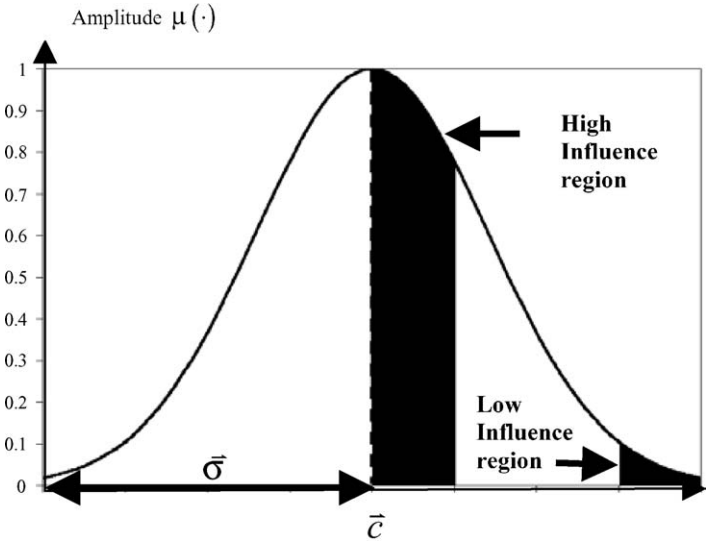

Fig. 2. Gaussian basis function used as expertise domain gating module.

sufficient to approximate the desired function within the expertise domain. In order to avoid such a redundancy, expertise level gating function is introduced to denote the importance of the local expert model so that the unimportant local expert can be deleted. Each expertise domain is multiplied by a weighting factor α_i known as expertise level function. It is obvious that α_i must also be a non-decreasing function of an adjustable parameter β_k. Smoothness of expertise function is also required because it is necessary for the gradient-based learning method. In addition, function α_k must be nonlinear because otherwise, it can be absorbed by local model function. To satisfy these requirements, the weighting factor α_k is defined as a logistic sigmoid function so that α_k is bounded ($0 < \alpha_k < 1$) for any real value of β_k where β_k is an adjustable parameter:

$$\alpha_k = \frac{1}{1 + e^{-\beta_k}} \tag{3}$$

The expertise level function is then multiplied by expertise domain function. Since the value of α_k is bounded ($0 < \alpha_k < 1$), the peak value of expertise domain (Gaussian basis function) $\mu_k(d(\vec{x}, \vec{c}_k, \vec{\sigma}_k))$ is gated by a weighting factor α_k. Therefore, the peak value of expertise domains varies according to the changes of β_k; in accordance with the importance of their contribution to the overall network output. Fig. 3 shows the effect of varying control parameter β on the peak value of expertise domain. In IGMN, the initial value of β is always set to zero and it is subject to error-based learning. The slope of expertise level function $\alpha(\beta)$ is greatest in the mid-range in the vicinity of $\alpha(0) = 0.5$. Near both extremes, the slope diminishes asymptotically toward zero as depicted in Fig. 4. Therefore, the expertise level value is sensitive to the onset of learning, which exhibits fast learning capability at the initial stage. This is closely similar to the learning curve of a human being.

2.3. Partition of unity

For modeling tasks the expertise domain (Gaussian basis functions) should form a partition of unity for the input space, i.e. at any point in the input space, the sum of all expertise domains should be unity. This is an essential requirement for the network to be able to globally approximate any desired function as complex as the local expert model. The partition of unity also ensures that any variation in output over the input space is due only to the local expert model. Therefore, the gated expertise domains are normalized to achieve the partition of unity:

$$\widehat{\Phi}(\cdot) = \frac{\alpha_k \cdot \mu_k(d(\vec{x}, \vec{c}_k, \vec{\sigma}_k))}{\sum_{k=1}^{m} \alpha_k \cdot \mu_k(d(\vec{x}, \vec{c}_k, \vec{\sigma}_k))} \tag{4}$$

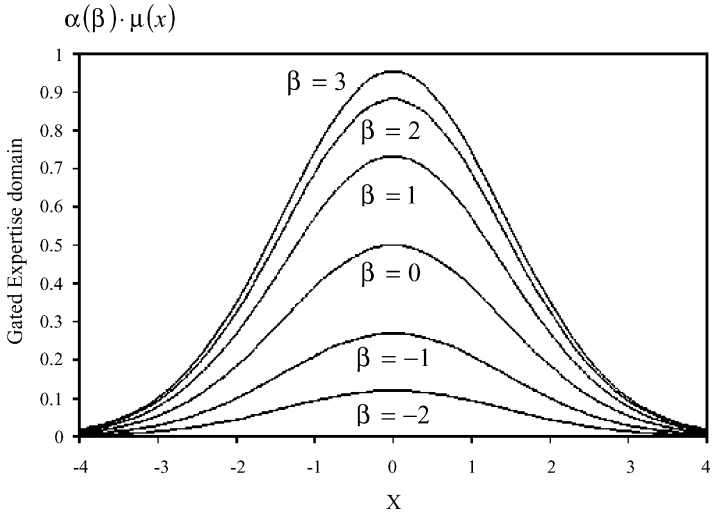

Fig. 3. β is the control parameter of expertise level function, $\alpha(\beta)$. The modifying value of β has the effect of suppressing the expertise domain. It is assumed the Gaussian function centered at $c = 0$ and the Gaussian width $\sigma = 1$.

where $\mu_k(d(\vec{x}, \vec{c}_k, \vec{\sigma}_k))$ is the general unnormalised expertise domain, therefore the m normalized expertise domains $\widehat{\Phi}_k(\cdot)$ sum to unity:

$$\sum_{k=1}^{m} \widehat{\Phi}_k(\cdot) = 1 \tag{5}$$

Normalization is very important for IGMN, often making the IGMN model less sensitive to the poor choice of Gaussian basis function parameters [9].

2.4. Local expert model

Since IGMN uses Gaussian function as expertise domain, it has an intrinsic feature of locality. It faces the same limitation in radial basis function network (RBF). The problem with RBF networks is that the

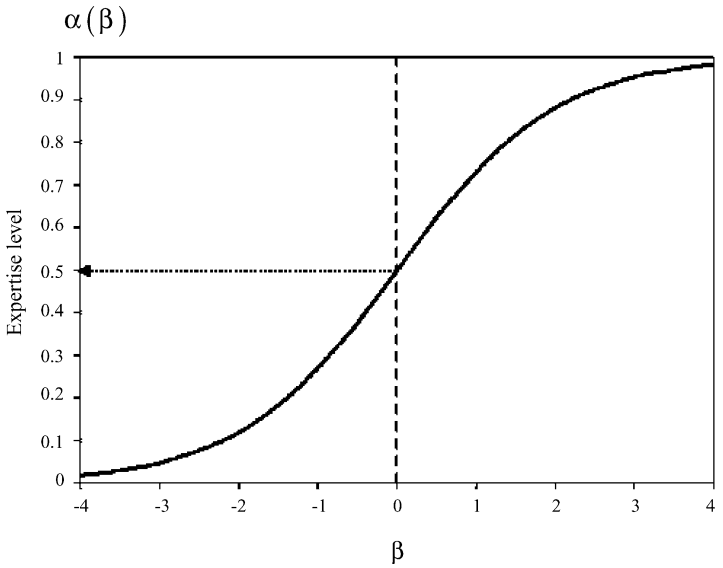

Fig. 4. The expertise level function of local experts.

crudeness of the piecewise constant model forces the network to use large number of basis functions to approximate a given function. This is especially severe for higher dimensional problems. It is therefore important to be able to profit from the local nature of the expertise domain while not requiring too many local experts. This implies that expertise domains should be associated with more powerful representations than piecewise constant models, so that a smaller number of local models could cover larger areas of the input space while achieving the desired modeling accuracy. Linear models, although very restricted in their representational ability have proved to be very useful for a large range of problems. This is due to their simple representation, their ease of interpretability, and their robustness to noisy or missing data. It makes sense therefore to include the ability to at least be able to form a linear model within the expertise domain. The linear model can be expressed as below:

$$\theta_k(\vec{x}) = \vec{x} \cdot \vec{b}_k + b_{0,k} \tag{6}$$

where $\vec{b}_k$ and $b_{0,k}$ are adjustable parameters of the local expert and $\vec{x}$ is the input vector.

Each local linear model is associated with the normalized gated expertise domain. The overall output of the IGMN can be described as given below:

$$\hat{y} = \sum_{k=1}^{m} \widehat{\Phi}_k(\cdot) \cdot \theta_k(\vec{x}) \tag{7}$$

The contribution of each local expert model is determined by validity of the input data to the expertise domain. The magnitude of contribution is further gated by expertise level. In general, the expertise level can be a good indicator of local expert redundancy. On the other hand, the overlapping of expertise domain allows the smooth combination of local expert outputs to produce a smooth approximation. The trained network structure can be viewed as a decomposition of the complex, nonlinear system into a set of locally active sub-models which are then smoothly integrated by their associated expertise domain and expertise level.

3. IGMN algorithm

IGMN uses incremental learning for the structural and parametric learning. IGMN is presumed to work under an environment that consists of a finite number of training examples although it might be possible to extend IGMN capability to work under an environment with infinite data stream. This extension is possible if the learning algorithm is fast enough to learn the training pattern within a single-pass. For a finite training examples environment, this single-pass learning requirement is relaxed and cyclic or repetitive learning is allowed [13]. The learning process of IGMN requires allocation of new local experts depending on the input novelty as well as cyclic and recursive adaptation of network. The novelty of the input data is tested using certainty factor that is coupled with sliding window root mean squared error (RMS) criterion. The incremental learning algorithm of IGMN is presented as follows:

- For each training pattern $(\vec{x}(t), y(t))$
  - Compute the overall IGMN output using Eq. (7).
  - Perform competitive Hebbian learning (CHL) learning [16]. The principle of CHL is for each input signal the best-matching local expert and the second-best matching expert are connected by a neighborhood edge.
  - Calculate the parameters required in the insertion criteria such as certain factor, CF(t) and sliding window RMS, $e_{\text{rms}}(t)$.
  - Apply the insertion criteria for adding new local expert:
    - If CF(t) $\leq$ CF cutoff threshold, δ_{cf} and $e_{\text{rms}}(t)$ $\geq e_{\text{rms}}(t-1)$ then allocate a new local expert.
    - Else adjust the IGMN parameters using momentum-based gradient descent algorithm and forgetting weighted recursive least squares (FWRLSs) algorithm.
  - For every δ_{delete} number of learning steps, check the criterion for deleting redundant local experts and removing the obsolete neighborhood edges with age exceeding age_{max}. They are identified by the fact that they are too old, which means CHL has not detected any correlated activity in age_{max} steps. This procedure allows a continuous update of the neighborhood information in the network.
- End For.

The learning algorithm is detailed as follows.

The network begins with two local experts that are initialized randomly. As observations are received some of the input data is taken as the new local experts. The following two criteria decide whether a new local expert should be inserted into IGMN:

$$\mathrm{CF}(t) \leq \delta_{\mathrm{cf}},$$
$$e_{\mathrm{rms}}(t) = \sqrt{\frac{\sum_{i=t-(t_{\mathrm{w}}-1)}^{t} [y(i) - \hat{y}(i)]^2}{t_{\mathrm{w}}}} \geq e_{\mathrm{rms}}(t-1) \tag{8}$$

The first criterion is a novelty measurement for the incoming input data whereas the second criterion is the sliding RMS for IGMN prediction. The parameter δ_{cf} is defined as certainty factor cut-off value, which indicates the tolerance range of IGMN prediction uncertainty. The certainty factor CF(t) is generated by IGMN networks using the expertise domain $\mu_k(d(\vec{x}, \vec{c}_k, \vec{\sigma}_k))$ and is updated recursively. In this scheme, for each input pattern, the activation value, $\mu_k(d(\vec{x}, \vec{c}_k, \vec{\sigma}_k))$ of each expertise domain is computed. These activation values are then ranked in descending order: $\mu_1(\cdot) > \mu_2(\cdot) > \mu_3(\cdot) > \cdots > \mu_k(\cdot) > \cdots > \mu_m(\cdot)$ where m is the number of local experts. The certainty factor CF for each input pattern is computed recursively according to the equation $\mathrm{CF}_k = \mathrm{CF}_{k-1} + (1 - \mathrm{CF}_{k-1})\mu_k(\cdot)$ for $k = 1, 2, 3, \ldots, m$ where $\mathrm{CF}_0 = 0$ and $\mathrm{CF}(t) = \mathrm{CF}_m$ is obtained. $\mathrm{CF}(t) < \delta_{\mathrm{cf}}$ means the IGMN prediction is unreliable and uncertain. The parameter t_{w} defines the number of past IGMN prediction errors to be included in sliding window RMS calculation. $e_{\mathrm{rms}}(t) \geq e_{\mathrm{rms}}(t-1)$ indicates the parameter learning of IGMN is not sufficient to improve the IGMN prediction accuracy and therefore a new local expert is recommended.

In order to allocate a new local expert, the criteria in Eq. (8) have to be fulfilled. The parameters of the new local expert are initialized as follows:

$$\vec{c}_{\mathrm{new}} = \vec{x}(t), \qquad \vec{\sigma}_{\mathrm{new}} = \frac{\sum_{k=1}^{k=\mathrm{Nb}} \|\vec{c}_k - \vec{c}_{\mathrm{w}_1}\|}{\mathrm{Nb}},$$
$$\beta_{\mathrm{new}} = 0, \qquad \psi_{\mathrm{new}} = 0 \tag{9}$$

The new local expert is associated with a Gaussian expertise domain, $\mu_{\mathrm{new}}(\cdot) = \mathrm{e}^{-|\vec{x}-\vec{c}_{\mathrm{new}}|^{\mathrm{T}} \vec{\sigma}_{\mathrm{new}}^{-1} |\vec{x}-\vec{c}_{\mathrm{new}}|}$. $\vec{c}_{\mathrm{w}_1}$ is the center of a local expert whose distance from $\vec{x}(t)$ is the nearest among those of all the other local experts. The spread of new expertise domain is calculated as the average distance, Nb of neighborhood local experts from $\vec{c}_{\mathrm{w}_1}$. The new expertise domain spread $\vec{\sigma}_{\mathrm{new}}$ is approximated based on the average of neighborhood distance from $\vec{c}_{\mathrm{w}_1}$ since there is no neighborhood edge emanated from $\vec{c}_{\mathrm{new}}$ yet. β_{new} is the expertise level parameter of $\alpha_{\mathrm{new}} = \frac{1}{1+\mathrm{e}^{-\beta_{\mathrm{new}}}}$. β_{new} is initialized as zero so that the expert level of the new local expert is set to its medium value 0.5. For a I dimension input pattern, the local expert model is defined as a linear model shown as follows:

$$\theta_{\mathrm{new}}(\vec{x}) = b_0 + b_1 x_1 + b_2 x_2 + b_3 x_3 + \cdots + b_I x_I \tag{10}$$

Eq. (10) can be written in vector form as follows:

$$\theta_{\mathrm{new}}(\vec{x}) = \vec{x}^{\mathrm{T}} \cdot \vec{b}_{\mathrm{new}} + b_{0,\mathrm{new}} = \tilde{x}^{\mathrm{T}} \cdot \vec{\zeta}_{\mathrm{new}} \tag{11}$$

where $\tilde{x} = [\vec{x}, 1], \vec{\zeta}_{\mathrm{new}}$ is the new local experts' parameter vector.

If the criteria in Eq. (8) are not fulfilled then the expertise domain center $\vec{c}_k$, the expertise domain spread $\vec{\sigma}_k$ and the expertise level β_k are learned using momentum-based gradient descent method where η_c, η_σ, η_α are learning rates and $\eta_{c_{\mathrm{M}}}$, $\eta_{\sigma_{\mathrm{M}}}$, $\eta_{\alpha_{\mathrm{M}}}$ are momentum rates:

$$\vec{c}_k(t) = \vec{c}_k(t-1) + \eta_c \cdot \left(-\frac{\partial E(t-1)}{\partial \vec{c}_k(t-1)} \right) + \eta_{c_{\mathrm{M}}} \cdot (\vec{c}_k(t-1) - \vec{c}_k(t-2)) \tag{12}$$

$$\vec{\sigma}_k(t) = \vec{\sigma}_k(t-1) + \eta_\sigma \cdot \left(-\frac{\partial E(t-1)}{\partial \vec{\sigma}_k(t-1)} \right) + \eta_{\sigma_{\mathrm{M}}} \cdot (\vec{\sigma}_k(t-1) - \vec{\sigma}_k(t-2)) \tag{13}$$

$$\beta_k(t) = \beta_k(t-1) + \eta_\alpha \left(-\frac{\partial E(t-1)}{\partial \beta_k(t-1)} \right) + \eta_{\beta_{\mathrm{M}}} (\beta_k(t-1) - \beta_k(t-2)) \tag{14}$$

The local expert model parameters for the next time step are updated using forgetting weighted recursive least squares algorithm (FWRLS) [8].

Given a training pattern $(\vec{x}, y)$, the incremental update of $\vec{\zeta}_k$ yields:

$$\vec{\zeta}_k(t+1) = \vec{\zeta}_k(t) + \mu_k(\cdot) \cdot \vec{P}_k(t+1) \cdot \vec{\Theta}_k(t) \cdot e(t) \tag{15}$$

where prediction error is calculated as $e(t) = y(t) - \hat{y}(t)$ and the forgetting factor ($0 < \lambda < 1$).

$$\vec{P}_k(t+1) = \frac{1}{\lambda} \cdot \left[\vec{P}_k(t) - \frac{\vec{P}_k(t) \cdot \vec{\Theta}_k(t) \cdot \vec{\Theta}_k^{\mathrm{T}}(t) \cdot \vec{P}_k(t)}{\frac{\lambda}{\mu(\cdot)_k} + \vec{\Theta}_k^{\mathrm{T}}(t) \vec{P}(t) \vec{\Theta}_k(t)} \right] \tag{16}$$

where $\vec{P}_k(t+1)$ is the covariance matrix.

$$\vec{\Theta}_k = \widehat{\Phi}_k(\cdot) \cdot \vec{x}^{\mathrm{T}}$$

where

$$\widehat{\Phi}_k(\cdot) = \frac{\alpha_k \cdot \mu_k(d(\vec{x}, \vec{c}_k, \vec{\sigma}_k))}{\sum_{k=1}^{m} \alpha_k \cdot \mu_k(d(\overrightarrow{x}; \overrightarrow{c}_k, \overrightarrow{\sigma}_k))} = \frac{\frac{1}{1+\mathrm{e}^{-\beta_i}} \cdot \mathrm{e}^{-|\vec{x}-\vec{c}_k|^{\mathrm{T}} \vec{\sigma}_k^{-1} |\vec{x}-\vec{c}_k|}}{\sum_{i=1}^{m} \frac{1}{1+\mathrm{e}^{-\beta_i}} \cdot \mathrm{e}^{-|\vec{x}-\vec{c}_k|^{\mathrm{T}} \vec{\sigma}_k^{-1} |\vec{x}-\vec{c}_k|}}$$

If the forgetting factor λ is small, then recent data are weighted more and the algorithm is more capable of tracking time-varying parameters. However, at the same time the estimators may also fluctuate to reflect noise and disturbance.

For δ_{delete} consecutive learning steps, if the normalized expertise level of local expertise model is smaller than pruning threshold, δ_{prune} according to Eq. (17):

$$\gamma_k = \frac{\alpha_k}{\sum_{k=1}^{m} \alpha_k} < \delta_{\mathrm{prune}} \tag{17}$$

This indicates the kth local expert is not important and can be deleted.

4. Stochastic modeling using IGMN

This section is devoted to the study of stochastic modeling using IGMN. It has been pointed out that stochastic fluctuations in the training data provide one way of introducing stochasticity into artificial neural networks [11]. Although artificial neural networks have been claimed to be robust against noisy data [10], a substantial amount of noise in the training data often confuses the networks. Most work on noise in artificial neural networks assumes that the networks are operated in an ideal Gaussian noise environment. However, a realistic model for the stochastic noise present in the training set should not be restricted to the assumption of ideal Gaussian distribution; instead, the distribution of the noise is often unknown or highly disturbed and thereby more seriously contaminated data such as outliers may occur.

For IGMN, the Gaussian function and the least mean squares (LMSs) criterion are selected as the basis function of network and the cost function, respectively. The network iteratively adjusts network parameters by minimizing the LMS criterion according to gradient descent and forgetting weighted recursive least squares method (FWRLS). It is well known that the LMS approach provides an optimal solution with minimum variance under the assumption that error term is Gaussian and independently and identically distributed with zero mean and unknown standard deviation [12]. This assumption usually fails to hold in real-world applications since either the error distribution is unknown or the training data collected are contaminated by non-Gaussian noise with much heavier tails such as Cauchy noise, in which some of the data points fall far away from the majority of the data; gross errors or outliers are encountered. The gross errors may deteriorate the reliability of IGMN since IGMN acts as an interpolative network in nature in an attempt to compensate for the outlying data with least squared residuals in the output space.

The residuals in the output space are used to fine-tune the network through a cost function. In the following, several choices of this cost function and their effect on the performance of IGMN is studied. This analysis also suggests a solution to the problem of outliers so that the IGMN model may produce a reliable answer when the training data is contaminated.

4.1. Cost function

When trained with data that contain gross errors or outliers, a neural network model obtained through the use of the LMS method often becomes inaccurate. For illustration, let us consider a neural network with one output unit y. The result can easily be generalized to networks with multiple outputs. The goal of a performance-oriented learning algorithm is to minimize a cost function of the form:

$$E = \frac{1}{N} \sum_{i=1}^{N} \rho(r(i)) \tag{18}$$

where the error function $\rho(\cdot)$ is symmetric and continuous, $r(i) = y_{\mathrm{d}}(i) - y(i)$ is the residual of pattern i with

target $y_d(i)$ and N is the number of training patterns. If $\vec{\omega}$ is the modifiable network parameter vector, then differentiating Eq. (18) with respect to $\vec{\omega}$ yields:

$$\frac{\partial E}{\partial \vec{\omega}} = \frac{1}{N}\sum_{i=1}^{N}\psi(r(i))\frac{\partial r(i)}{\partial \vec{\omega}} \tag{19}$$

which is equal to zero at the minimum, where

$$\psi(r(i)) = \frac{\partial \rho(r(i))}{\partial r(i)}$$

is the influence function. If the right-hand side of Eq. (19) is interpreted as a weighted sum of $\frac{\partial r(i)}{\partial \vec{\omega}}$, then $\psi(r(i))$ represents the weight or influence of pattern i.

The process by which a small number of outliers can have a large impact on the results can be understood through the analysis of the method's influence function.

4.1.1. Least mean squares (LMSs)

Using the notations defined above, the LMS method is realized by setting the cost function:

$$\rho(r) = \frac{1}{2}r^2 \tag{20}$$

The influence function then becomes:

$$\psi(r(i)) = \frac{\partial \rho(r(i))}{\partial r(i)} = r(i) \tag{21}$$

The influence function demonstrates that the more deviant a point from the model, the greater its weights in Eq. (19). Specifically, an outlier, which by definition has a large residual $r(i)$, can create large inaccuracy in the model. Therefore, the outliers can alter the accuracy of a model trained using the LMS method. One alternative for controlling the damage caused by outliers is to bound the value of the influence function. Literature in robust statistics provides in depth discussions of a wide variety of bounded influence functions [6,14,15]. In the following section two influence functions namely robust least mean squares (RLMSs) [5] and least mean log squares [12] are utilized and evaluated to improve the robustness of IGMN against outliers.

4.1.2. Robust least mean squares (RLMSs)

The IGMN algorithm uses the least mean squares (LMSs) method may not converge to an optimal solution. In order to alleviate the outlier problem, robust least mean squares (RLMSs) [5] is used as the cost function of the networks. Robust least mean squares (RLMSs) is designed to be stable under minor noise perturbation and robust against gross errors in the data caused by outliers. The influence function [5] is represented as

$$\psi(r(i)) = re^{-r(i)^2/2\sigma} \tag{22}$$

and the corresponding cost function is therefore be obtained as

$$\rho(r(i)) = \sigma(1 - e^{-r(i)^2/2\sigma}) \tag{23}$$

where

$$\frac{\partial \rho(r(i))}{\partial r(i)} = \psi(r(i))$$

Fig. 5a illustrates the influence curve $\psi(r(i))$. It is clear that σ controls the position and the magnitude of the peak in each curve. The influence function decreases monotonically to zero as $r(i) \to \pm\infty$ and they have the effect of suppressing the large residue error $r(i)$ caused by outliers. On the other hand, the derived robust objective function shown in Fig. 5b has decreasing gradient when $r(i) \to \pm\infty$. This means the large residue error $r(i)$ is constrained to a limit.

The σ here is adjusted adaptively during a training procedure to improve the accuracy of approximation using the following Eq. (24).

When the value of objective function changes rapidly, $|\rho(r(i)) - \rho(r(i-1))| > \delta_{\text{upper}}$, reduce the confidence interval of the residue by finding new1 cutoff points according to

$$\sigma_{\text{new}} = \begin{cases} \sigma_{\text{old}} e_d & \text{if } \sigma_{\text{new}} > \sigma_L \\ \sigma_L & \text{otherwise} \end{cases} \tag{24}$$

where e_d is the decreasing rate and σ_L is the lower bound of the confidence interval of residual as depicted in Fig. 5a. The residue value $r(i)$ caused by outliers will be suppressed to zero, when it falls outside the confidence interval. $\rho(r(i-1))$ is $\rho(r(i))$ of the previous step. The simulation study in the following section sets $e_d = 0.05$, $\sigma_L = 0.2$. The cutoff points are initialized as $\sigma_{\text{ini}} = 0.5$ and $\delta_{\text{upper}} = 0.5$.

4.1.3. Least mean log squares (LMLSs)

In this method, the influence function is defined as

$$\psi(r(i)) = \frac{r(i)}{1 + \frac{1}{2}r(i)^2} \tag{25}$$

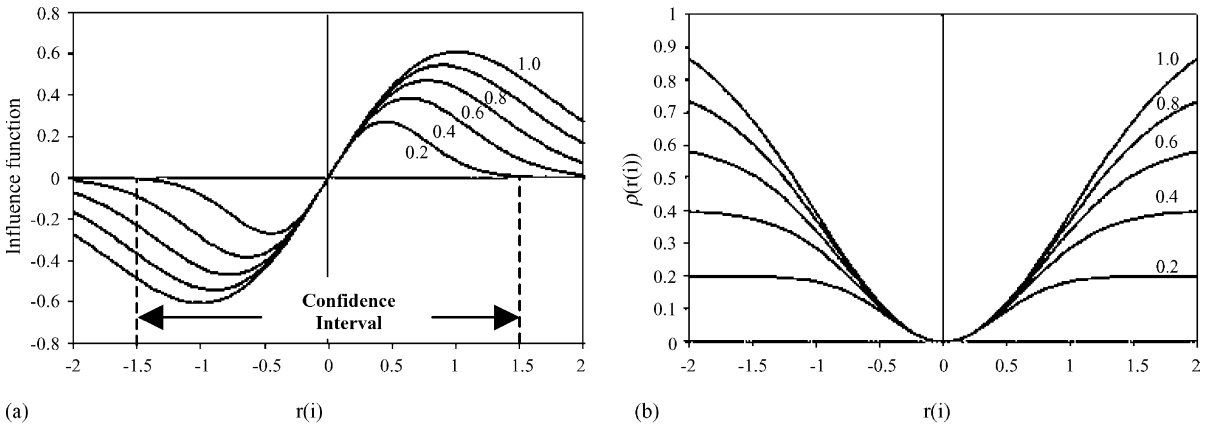

Fig. 5. (a) Influence function $\psi(r(i))$ curves and (b) the corresponding robust objective function $\rho(r(i))$ for different value of σ.

It is chosen due to its simplicity and compatibility with most learning algorithms. In particular, any learning algorithm that uses the LMS error function $\rho(r(i)) = \frac{1}{2}r(i)^2$ can be easily modified to use the new error function:

$$\rho(r(i)) = \log\left(1 + \frac{1}{2}r(i)^2\right) \tag{26}$$

It is known as least mean log squares (LMLSs) method. The LMS and LMLS influence functions are plotted in Fig. 6 for comparison. For small residuals, both influence functions have approximately the same values. As the residual increases, the magnitude of LMLS influence function reaches its maximum and start to decrease. The impact of outliers on the LMLS method is thus limited. In fact, for very large residuals, the outliers have no effect at all due to

$$\lim_{r(i)\to 0} \psi_{\mathrm{LMLS}}(r(i)) = \lim_{r(i)\to\infty} \frac{r(i)}{1 + \frac{1}{2}r(i)^2} = 0 \tag{27}$$

Hence, LMLS is robust against gross errors in the data set. The LMS method, on the other hand, is extremely sensitive to gross errors. As the residual increases, its influence increases proportionally. A single large residual from an outlier can outweigh the remaining small residuals and prevent the model from converging to the correct function.

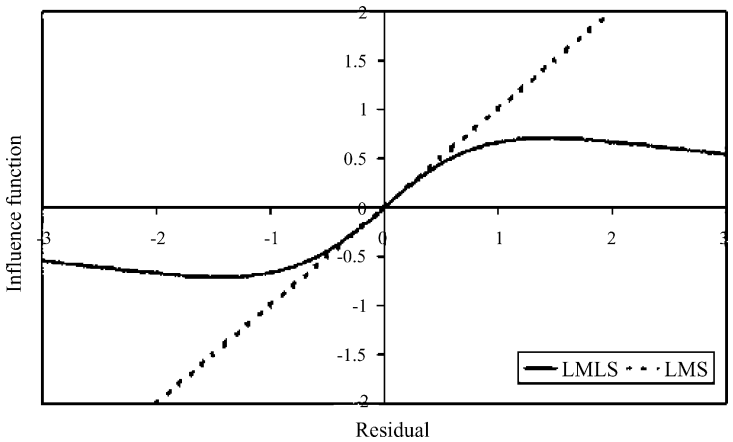

Fig. 6. The influence functions for LMS and LMLS methods.

Table 1
Parameter setting of incremental GMN for robust learning evaluation

Certainty factor, δ_{cf}	0.6
Pruning threshold, δ_{prune}	0.01
Adaptation parameter, η_c	0.01
Adaptation parameter, η_σ	0.01
Adaptation parameter, η_β	0.98
Momentum term, η_M	0.1
Forgetting factor, λ	0.998
Number of adaptation steps before pruning, δ_{delete}	100
Edge removal threshold, age_{max}	500

4.2. Performance comparison between LMS, robust LMS and LMLS using Gaussian noise model

To compare the performance of the LMS, Robust LMS and LMLS methods, every controllable parameter such as network learning parameters and learning algorithm are kept exactly the same. The learning parameters selection of IGMN is given in Table 1. For simplicity, the IGMN is used to approximate a one input-one output function as follows:

$$y = x^{2/3} \tag{28}$$

using the IGMN algorithm. The simulations are run using data with three types of error distribution:

1. Very high-quality data with absolutely no error to represent artificial data for reference.
2. High-quality data with small Gaussian noise to represent data collected from a carefully designed experiment.
3. Data with small Gaussian noise and 5% gross error to represent the majority of real data.

A total of 500 training patterns are generated by evenly sampling the independent variable in the range $[-2, 2]$ and using Eq. (28) to calculate the dependent variable. These patterns are then used in the first simulation. For the second simulation, error from $N(0.0, 0.1)$ Gaussian distribution with mean of 0.0 and a standard deviation of 0.1, is added to the independent variable. The gross error model:

$$F = (1 - \varepsilon)G + \varepsilon H \tag{29}$$

where F is the error distribution and G and H are probability distributions that occur with probability $(1 - \varepsilon)$ and ε, respectively [14,15], is added to the independent variable in the third simulation. The values used in the gross error model are

$$\varepsilon = 0.05, \qquad G \sim N(0.0, 0.1), \quad \text{and}$$
$$H \sim N(10, 0.1)$$

4.2.1. Simulation results

Each of the three data sets is trained with LMS, robust LMS and LMLS methods for 50 epochs. The results are summarized in the figures below. Each figure contains the original function for reference, the data used in training, the approximated function using LMS, robust LMS and LMLS method.

The first experimental data set is presented in Fig. 7a. In the second experiment, Gaussian noise is added to the training data as depicted in Fig. 7b. The third experiment training data contains gross errors as depicted in Fig. 7c. The root-mean-squared error (RMS) is used to quantify the performance of the networks. RMS is defined as

$$\text{RMS} = \sqrt{\frac{\sum_{i=1}^{N}(y_d(i) - y(i))}{N}} \tag{30}$$

where the target $y_d(i)$ is the actual value of the function at $x(i)$ and $y(i)$ is the output of the network given $x(i)$ as its input.

Table 2 shows that in experiments 1 and 2 where there are no outliers, there is practically no difference in the performance of the networks trained with LMS, robust LMS and LMLS methods. When outliers are present as in experiment 3, however, the network trained with LMS method fails to converge while the RMS error for the robust LMS and LMLS network increase only slightly. This shows that the LMLS and robust LMS methods are superior. The function approximated by the network trained with LMS method is inaccurate, while the network trained with the LMLS and robust LMS method produces a good approximation to the original function.

Table 2
RMS values of IGMN trained with LMS, robust LMS and LMLS

Experiment	LMS	Robust LMS	LMLS
1	0.0241	0.0269	0.0238
2	0.0568	0.0589	0.0567
3	0.6020	0.0319	0.0401

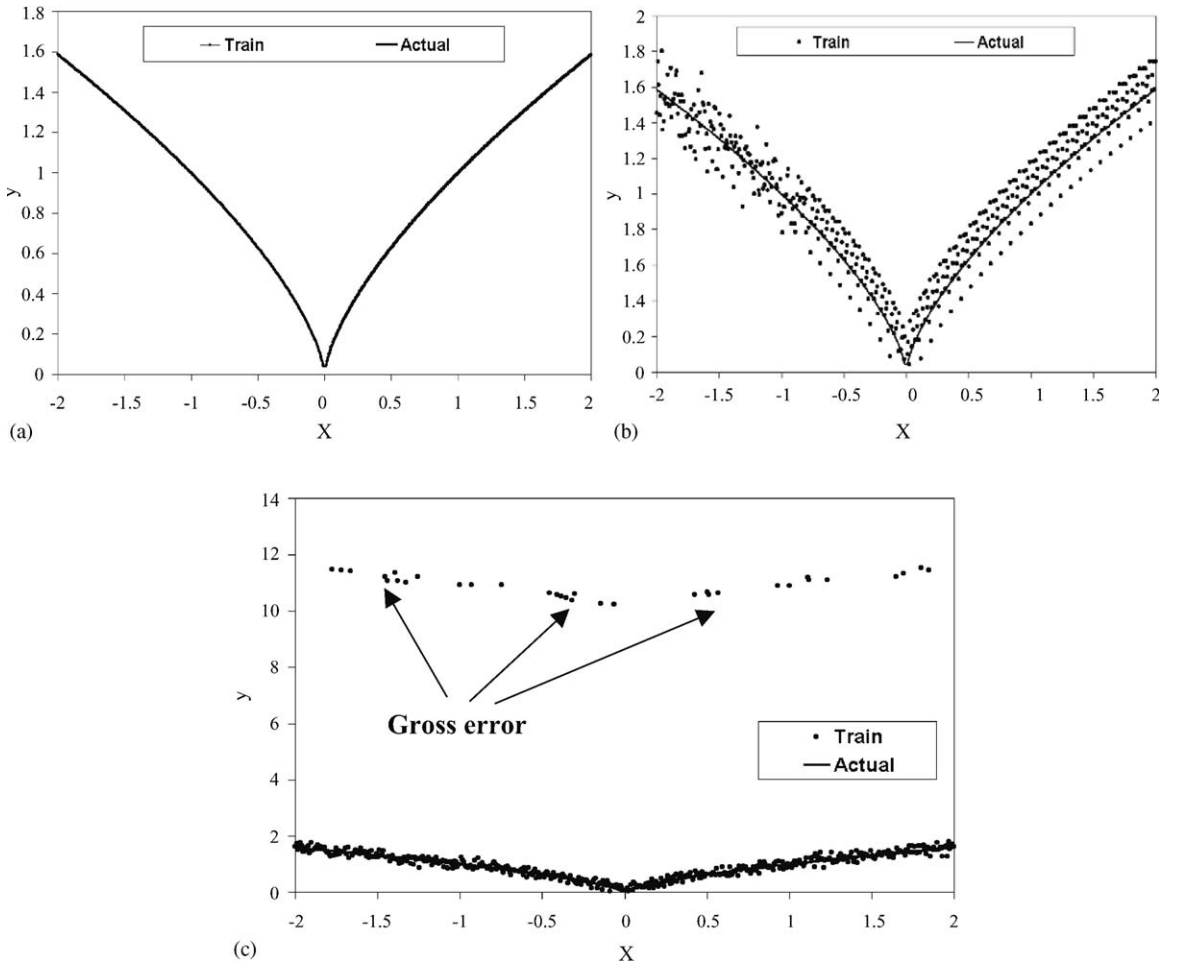

Fig. 7. (a) The actual function (which intersects the origin) and the training patterns used in experiment 1. (b) The actual and the training patterns used in experiment 2. (c) The actual function and the training patterns used in experiment 3.

4.2.2. Training analysis

Eq. (31) shows that the learning on the network parameters due to an error directly proportional to the influence function used. Since $|\Psi_{\mathrm{LMS}}(r(i))| > |\Psi_{\mathrm{LMLS}}(r(i))|$ and $|\Psi_{\mathrm{LMS}}(r(i))| > |\Psi_{\mathrm{robust\,LMS}}(r(i))|$ due to

$$|r(i)| > \left| \frac{r(i)}{1 + \frac{1}{2} r(i)^2} \right| \quad \text{for } r(i) \neq 0 \tag{31}$$

and plotted in Fig. 8, the LMS method is more responsive to the error and therefore should be able to reduce the error faster than the LMLS method. This would mean that network using the LMS method learns faster than network using the LMLS method as is observed in the network training histories. Since most variations in the error measures occur during the early stages in training, only the first few epochs of the training histories are shown. For comparison RMS are calculated for each method during training.

Error histories of the training using exact data are plotted in Fig. 8a and error histories of the training using data containing small Gaussian noise are plotted in Fig. 8b. These two figures show that network training histories are qualitatively the same for both data sets. For the first few epochs, both Fig. 8a and b shows that the LMS method and LMLS method are more efficient than the Robust LMS method. This is

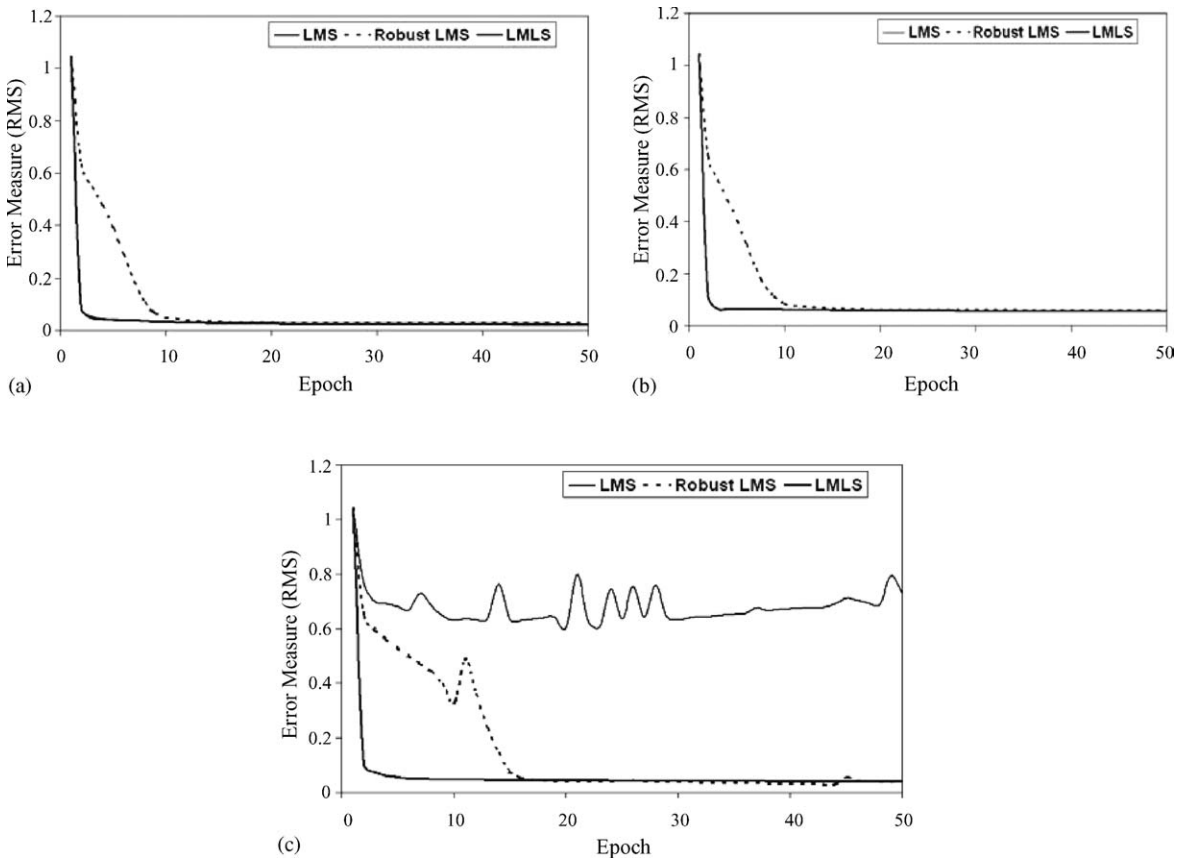

Fig. 8. (a) RMS values of IGMN using LMS, robust LMS and LMLS method during training using exact data. (b) RMS values of IGMN using LMS, robust LMS and LMLS method during training using Gaussian noise. (c) RMS values of IGMN using LMS, robust LMS and LMLS method during training using data with gross errors.

expected since a legitimate point with Gaussian noise error has a larger in influence in the LMS and LMLS than in the robust LMS method. This leads to a faster adaptation of the LMS model in the beginning. After a few epochs when all the errors have decreased to smaller values, however, the learning speed of the methods converge due to $\frac{r(i)}{1+\frac{1}{2}r(i)^2} \approx r(i)$ for small r(i).

Training histories for the networks using data containing gross errors are plotted in Fig. 8c. In terms of the RMS error measure, it clearly shows that LMLS method is significantly better than LMS method. Robust LMS is also superior to LMS method but the learning speed appears slower at the early stage of learning than LMLS method.

The experimental results above have demonstrated the robustness of the proposed learning methods. However, this study is limited to Gaussian noise. In real-world applications since either the error distribution is unknown or the training data collected are contaminated by non-Gaussian noise with much heavier tails such as occurring in Cauchy distribution in which gross errors or outliers are encountered. In the following section, IGMN is tested on data set contaminated with Cauchy noise and spiky outliers.

4.3. Performance comparison between LMS, robust LMS and LMLS using stochastic Cauchy noise model

The Hermite polynomial function is defined as

$$f(x) = 1.1(1 - x + 2x^2)\exp\left(-\frac{1}{2}x^2\right)$$

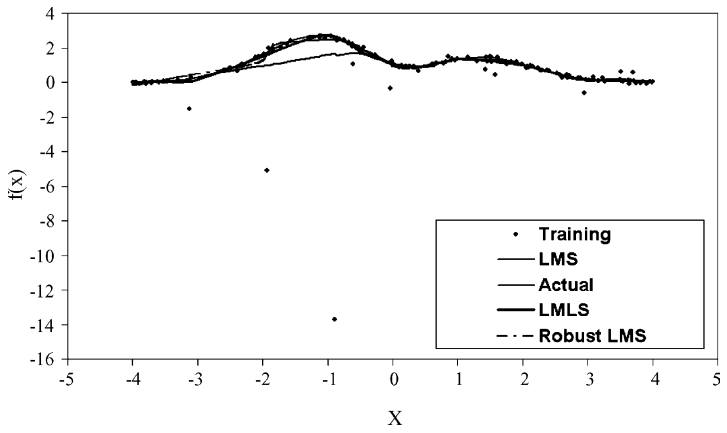

Fig. 9. The actual function of Hermite function, the training data with Cauchy distributed errors and the predictions of IGMN using LMS, robust LMS and LMLS.

One hundred and fifty training samples are obtained by random sampling of the interval [−4, 4]. The training data set is contaminated by stochastic errors according to the Cauchy distribution with location 0 and scale 0.1. Fig. 9 shows the original and the contaminated Hermite function. Fig. 10 shows that the LMLS method is superior. IGMN trained with the LMS method is inaccurate whereas IGMN trained with the robust LMS method exhibits oscillation at the early stage of learning even though convergence is achieved subsequently to almost similar solution as LMLS method. The convergence of the LMLS method is faster than the robust LMS method. This shows the competence of LMLS to learn Cauchy noise contaminated data.

4.4. Performance comparison between LMS, robust LMS and LMLS using synthetic outlier model

In order to investigate the impact of vertical outliers, four synthetic outliers are injected into the Hermite function. IGMN is trained using the contaminated data for 200 epochs. Fig. 11 shows the original and the contaminated Hermite function. Fig. 12 shows that the LMLS method is superior. IGMN trained with the LMS method is inaccurate and unstable. IGMN trained with the robust LMS method exhibits gross learning error at the early stage of learning even though convergence is achieved subsequently to almost similar solution as LMLS method. The convergence of the LMLS method

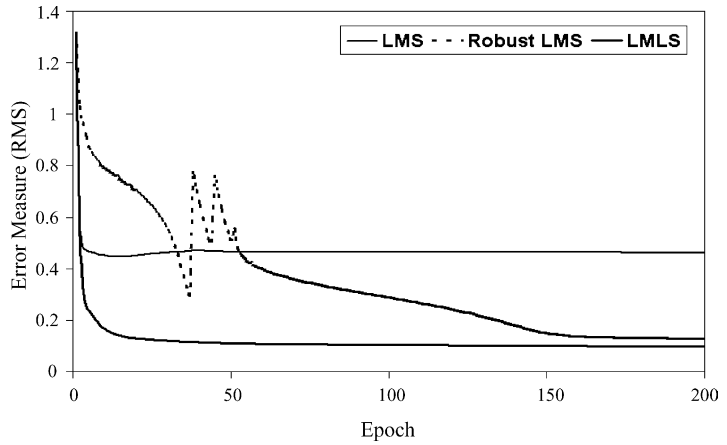

Fig. 10. RMS values of IGMN using LMS, robust LMS and LMLS method during training using synthetic outliers.

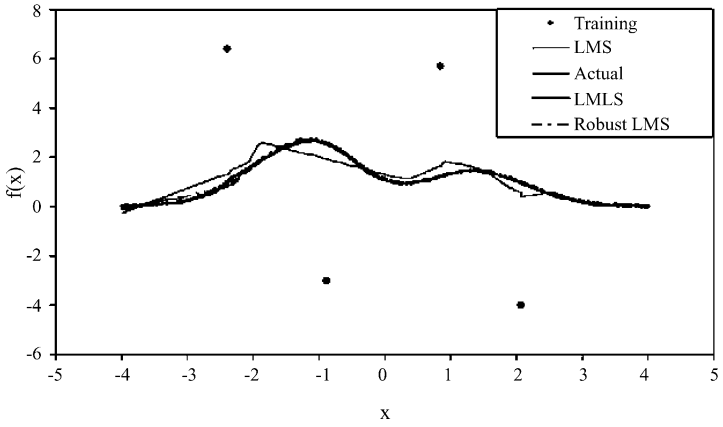

Fig. 11. The Hermite function contaminated by four synthetic outliers.

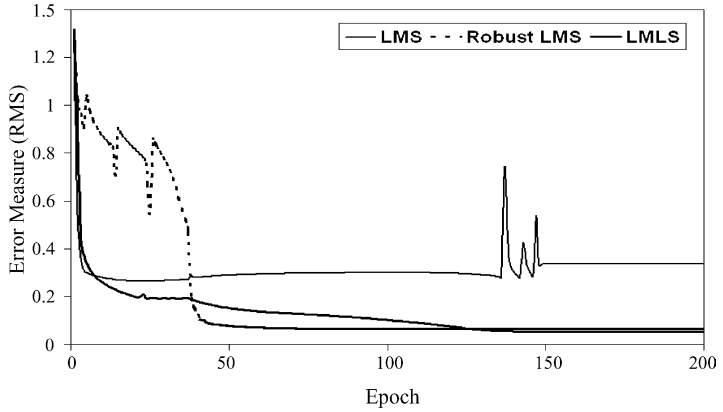

Fig. 12. RMS values of IGMN using LMS, robust LMS and LMLS method during training using synthetic outliers.

is more stable in learning even though the convergence is slower than the robust LMS method at early stage. This shows the feasibility of both the robust LMS and LMLS to learn training data containing spiky outliers.

In summary, LMLS error measure appeared to be the most favorable robust learning method due to its robustness to various type of noise either Gaussian or non-Gaussian noise and spiky outliers. In other words, IGMN may use LMLS as error measure to learn data set contains unknown noise or gross error. This method is parameter free. The implementation of this method is straightforward and readily applied to the existing IGMN learning method. This simple enhancement makes IGMN reliable with any real data.

5. Conclusions

This paper outlines two robust learning methods namely robust least mean squares (RLMSs) and least mean log squares (LMLSs) for incremental growing multi-experts network (IGMN). It is demonstrated that the model produced by the least mean squares (LMSs) method is inaccurate when the data contains outliers. The RLMS and LMLS methods, on the other hand, works well whether the data set is exact, contains Gaussian noises, Cauchy noise or even spiky outliers. In addition, various types of supervised learning algorithms can easily adopt LMLS, which is a free parameter method.

References

[1] C.K. Loo, M. Rajeswari, Redundant expert detection for a growing multi-experts network, in: Proceedings of the Regional Symposium on Quality and Automation, Penang, Malaysia, 2000, pp. 365–374.

[2] C.K. Loo, M. Rajeswari, Growing multi-experts network, in: Proceedings of the TENCON 2000, Kuala Lumpur, Malaysia, September 2000, pp. 472–478.

[3] C.K. Loo, M. Rajeswari, Growing multi-experts network for industrial control applications, in: Proceedings of the World Conference on Soft Computing in Industrial Applications, December 2000.

[4] K. Liano, A robust approach to supervised learning in neural network, in: Proceedings of the ICNN'94, vol. 1, 1994, pp. 513–516.

[5] C.-C. Lee, P.-C. Chung, J.-R. Tsai, C.-I. Chang, Robust radial basis function neural networks, IEEE Trans. Syst. Man Cybern. Part B: Cybern. 29 (6) (1999).

[6] F.R. Haempel, E.M. Ronchetti, P.J. Rousseeuw, W.A. Stahel, Robust Statistics, Wiley, New York, 1986.

[7] D.S. Chen, R.C. Jain, A robust back propagation learning algorithm for function approximation, IEEE Trans. Neural Networks 5 (3) (1994).

[8] L. Ljung, T. Soderstrom, Theory and Practice of Recursive Identification, MIT Press, Cambridge, MA, 1983, pp. 57–59.

[9] H.W. Werntges, Partitions of unity improve neural function approximation, in: Proceedings of the IEEE International Conference on Neural Networks, vol. 2, San Francisco, CA, 1993, pp. 914–918.

[10] S.-I. Amari, Mathematical foundations of neurocomputing, Proc. IEEE 78 (9) (1990) 1464–1480.

[11] S.-I. Amari, Information geometry of the EM and EM algorithm, Neural Networks 8 (9) (1995) 1379–1408.

[12] L. Kadir, Robust error measure for supervised neural network learning with outliers, IEEE Trans. Neural Network 7 (1) (1996) 246–250.

[13] T. Heskes, W. Wiegerinck, A theoretical comparison of batch-mode, on-line, cyclic, and almost-cyclic learning, IEEE Trans. Neural Network 7 (4) (1996) 919–925.

[14] P.J. Huber, Robust Statistics, Wiley, New York, 1981.

[15] P.J. Rousseeuw, A.M. Leroy, Robust Regression and Outlier Detection, Wiley, New York, 1987.

[16] T.M. Martinetz, Topology representing networks, Neural Networks 7 (3) (1994) 507–522.

[17] C.K. Loo, M. Rajeswari, M.V.C. Rao, Extrapolation detection and novelty-based node insertion for sequential growing multi-experts network, Appl. Soft Comput. 3 (2) (2003) 159–175.